

Propósito del Documento: Definir la arquitectura, el modelo de datos, la criptografía de firma de API y el flujo de integración de Zadarma SMS en el proyecto CRM Salvadora (`d:\tmp\antigraviti\crm_salvadora`). Permite confirmar reservas telefónicas realizadas por el agente de voz VAPI y solicitar el correo del cliente de manera no intrusiva.

1 Flujo Operativo y Captura Post-Llamada

Durante la llamada telefónica gestionada por VAPI, la reserva se ejecuta limpiamente con nombre y teléfono. Tras la confirmación, el agente de voz formula una pregunta contextual suave para el email. Simultáneamente, el CRM emite un SMS automático que actúa como canal de confirmación y captura secundaria:

PASO 1: LLAMADA
Reserva Telefónica
El usuario llama y VAPI ejecuta `reservar_cita` registrando contacto y cita en CRM.

PASO 2: VOZ
Email Opcional
La IA ofrece enviar el resumen por correo. Si el cliente duda, no bloquea la cita.

PASO 3: DISPARO
Trigger Zadarma SMS
El CRM dispara SMS vía Zadarma API con los detalles y mensaje de captura.

PASO 4: AUDITORÍA
Registro en DB
Se audita el costo, mensaje, teléfono y estado en tabla `zadarma_sms_respuesta`.

Plantilla de SMS Recomendada:
"¡Hola! Tu reserva de Yoga en Centro Salvadora está confirmada. Si deseas recibir los detalles de acceso y recordatorios por correo, respóndenos con tu email a este SMS o infórmalo en la recepción. ¡Te esperamos!"

2 Algoritmo Criptográfico de Zadarma API

La API de Zadarma exige autenticación mediante firma HMAC-SHA1 de los parámetros de consulta ordenados alfabéticamente y procesados con hash MD5 previo:

Especificaciones del Endpoint

URL Base	<code>https://api.zadarma.com</code>
Ruta API	<code>/v1/sms/send/</code>
Método	POST (URL Encoded)
Cabecera Auth	<code>Authorization: {api_key}:{signature}</code>

Eliminación de dependencias obsoletas: Se prescindió por completo de la tabla `accionescomplementarias` del script original. Los envíos se parametrizan directamente por llamada o por endpoints REST.

Algoritmo de Firma Criptográfica

```
1. Ordenar claves de params alfabéticamente:
   params = { message: "...", number: "34611..." }
2. Generar QueryString: "message=...&number=..."
3. MD5 de QueryString: md5_str = md5(queryString)
4. Cadena a firmar:
   toSign = "/v1/sms/send/" + queryString + md5_str
5. HMAC-SHA1 con api_secret:
   hmac_hex = hmac_sha1(toSign, api_secret).toHex()
6. Base64 de la cadena hexadecimal:
   signature = base64(hmac_hex)
7. Header: "Authorization: {api_key}:{signature}"
```

3 Modelo de Datos PostgreSQL

El CRM registra cada envío para control de costes, auditoría y trazabilidad en dos scripts DDL:

Tabla `zadarma_sms_respuesta`

```
CREATE TABLE IF NOT EXISTS zadarma_sms_respuesta (
  id SERIAL PRIMARY KEY,
  fecha TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  status VARCHAR(20) NOT NULL,      -- success / error
  messages INT DEFAULT 1,           -- cant. fragmentos
  cost NUMERIC(10, 4) DEFAULT 0,    -- coste en EUR
  currency VARCHAR(10) DEFAULT 'EUR',
  raw_response TEXT,                -- JSON crudo devuelto
  phone VARCHAR(30) NOT NULL,
  message TEXT NOT NULL,
  contact_id UUID REFERENCES contacts(id),
  call_id UUID REFERENCES calls(id),
  appointment_id UUID REFERENCES appointments(id)
);
```

Parametrización de API Keys (DDL)

```
-- 1. Extensión para vapi_accounts (CRM Voice settings):
ALTER TABLE vapi_accounts
ADD COLUMN IF NOT EXISTS "zadarmaApiKey" VARCHAR(100)
  DEFAULT '45dc42d6f22439899024',
ADD COLUMN IF NOT EXISTS "zadarmaApiSecret" VARCHAR(100)
  DEFAULT '34061190a934a453aa99',
ADD COLUMN IF NOT EXISTS "zadarmaSenderId" VARCHAR(50)
  DEFAULT NULL,
ADD COLUMN IF NOT EXISTS "zadarmaSmsEnabled" BOOLEAN
  DEFAULT TRUE;

-- 2. Variables de entorno de respaldo (.env):
-- ZADARMA_API_KEY=45dc42d6f22439899024
-- ZADARMA_API_SECRET=34061190a934a453aa99
```

4 Contenido del Paquete de Código Entregado

Todos los archivos listos para producción se encuentran en la carpeta `salvadora/caso1/integracion_sms_zadarma_crm/`:

Directorio / Archivo	Descripción	Destino o Modo de Uso
<code>csharp/Program.cs</code> <code>csharp/ZadarmaSms.csproj</code>	Programa .NET adaptado (compatible con .NET 8 / 9 / 10). Admite argumentos por línea de comandos o servidor HTTP autónomo en puerto 5005.	Ejecutable CLI / Microservicio: <code>dotnet run -- 34611... "Mensaje"</code> o <code>POST :5005/api/sms/send</code>
<code>nest_backend/zadarma-sms.service.ts</code>	Servicio TypeScript nativo para NestJS. Implementa firma criptográfica, llamada Axios a Zadarma y persistencia en TypeORM.	Copiar a <code>crm_salvadora/backend/src/sms/</code>
<code>nest_backend/zadarma-sms.controller.ts</code>	Controlador REST con endpoints de envío (<code>/api/sms/send</code>), consulta de logs (<code>/api/sms/logs</code>) y parametrización de keys (<code>/api/sms/config</code>).	Copiar a <code>crm_salvadora/backend/src/sms/</code>
<code>nest_backend/zadarma-sms-log.entity.ts</code>	Entidad TypeORM mapeada a la tabla <code>zadarma_sms_respuesta</code> con relaciones a Contacto y Cita.	Copiar a <code>crm_salvadora/backend/src/sms/</code>
<code>nest_backend/vapi-hook-integration.snippet.ts</code>	Código de ejemplo para inyectar el envío automático en <code>vapi-webhook.service.ts</code> al confirmar cita o en <code>end-of-call-report</code> .	Referencia para el desarrollador del CRM
<code>sql/*.sql</code>	Scripts DDL para creación de tablas y adición de columnas de configuración.	Ejecutar en la base de datos PostgreSQL de Salvadora
<code>tests/*.sh</code> / <code>tests/*.ps1</code>	Scripts de prueba automáticos mediante curl y PowerShell para verificar el funcionamiento.	Pruebas de humo e integración local

5 Contratos de la API REST de Envío SMS

`POST /api/sms/send` Envío de SMS puntual o masivo

Cuerpo de la Solicitud (JSON):

```
{
  "number": "34611223344",           // Requerido (con o sin +)
  "message": "Tu reserva de Yoga...", // Requerido
  "sender": "Salvadora",             // Opcional (ID alfanumérico)
  "contactId": "uuid-del-contacto",  // Opcional para auditoría
  "callId": "uuid-de-llamada",       // Opcional
  "appointmentId": "uuid-de-cita"    // Opcional
}
```

Respuesta de Éxito (HTTP 200):

```
{
  "success": true,
  "status": "success",
  "messages": 1,
  "cost": 0.045,
  "currency": "EUR",
  "rawResponse": {
    "status": "success",
    "messages": 1,
    "cost": "0.045",
    "currency": "EUR"
  },
  "logId": 142
}
```

`PATCH /api/sms/config` Parametrización dinámica de Credenciales

Cuerpo para Actualizar API Keys (JSON):

```
{
  "zadarmaApiKey": "45dc42d6f22439899024",
  "zadarmaApiSecret": "34061190a934a453aa99",
  "zadarmaSenderId": "Salvadora",
  "zadarmaSmsEnabled": true
}
```

Respuesta (HTTP 200):

```
{
  "success": true,
  "message": "Zadarma SMS configuration updated successfully"
}
```

`GET /api/sms/logs?limit=50` Auditoría histórica de SMS enviados

6 Integración con los Webhooks de VAPI en el CRM

En el backend del CRM (`src/vapi/vapi-webhook.service.ts`), el envío del SMS se vincula al ciclo de vida de la reserva telefónica. Se recomiendan dos estrategias no excluyentes:

Estrategia Inmediata (Herramienta `reservar_cita`)

En cuanto la herramienta de función responde afirmativamente, se dispara el SMS de manera asíncrona (sin bloquear el audio de respuesta de VAPI).

```
// En vapi-webhook.service.ts -> executeReservarCita():
if (appointmentCreated && customerPhone) {
  this.zadarmaSms.sendSms({
    number: customerPhone,
    message: `Centro Salvadora: Confirmamos tu cita de ` +
      `${serviceName} para el ${fechaLegible}. ` +
      `Respóndenos con tu email para enviarte el acceso.`,
    contactId: contact.id,
    appointmentId: appointment.id,
    callId: call.id,
  }).catch(err => this.logger.error('Error SMS:', err));
}
```

Estrategia Post-Llamada (Evento `end-of-call-report`)

Cuando el cliente cuelga el teléfono, VAPI envía el webhook de fin de llamada con el análisis y la duración. Es el momento idóneo para el SMS de bienvenida.

```
// En vapi-webhook.service.ts -> handleEndOfCallReport():
const call = await this.callsRepo.findOne({ where: { vapiCallId } });
if (call?.appointmentCreated && call.customerNumber) {
  await this.zadarmaSms.sendSms({
    number: call.customerNumber,
    message: `¡Gracias por tu llamada a Centro Salvadora! ` +
      `Tu cita está confirmada. Si deseas recibir ` +
      `el comprobante por email, respóndenos a este SMS.`,
    callId: call.id,
    contactId: call.contactId,
  });
}
```

7 Guía Paso a Paso de Despliegue para el Desarrollador

Verificación completada: El módulo C# fue compilado y verificado con .NET 10.0 con envío real exitoso hacia la API de Zadarma (HTTP 200). El backend de NestJS compila con 0 errores.

1. Aplicar la migración SQL en PostgreSQL:

Ejecutar `01_create_zadarmasmrespuesta.sql` y `02_alter_settings_and_vapi_accounts.sql` en la base de datos de producción o desarrollo del CRM.

2. Incorporar el módulo en NestJS:

Copiar la carpeta `nest_backend/` como `src/sms/` en `d:\tmp\antigraviti\crm_salvadora\backend\`.

Registrar `ZadarmaSmsModule` en `app.module.ts` o `vapi.module.ts`.

3. Parametrizar las Credenciales de Zadarma:

Acceder a `PATCH /api/sms/config` o configurar las variables de entorno `ZADARMA_API_KEY` y `ZADARMA_API_SECRET` en el archivo `.env`.

4. Conectar el Trigger en VapiWebhookService:

Seguir las instrucciones de `vapi-hook-integration.snippet.ts` para disparar el SMS tras reservas de clases o llamadas entrantes finalizadas.

5. Ejecutar la suite de pruebas:

Correr `./tests/test_sms.ps1` o `./tests/test_sms.curl.sh` indicando el número móvil de prueba.